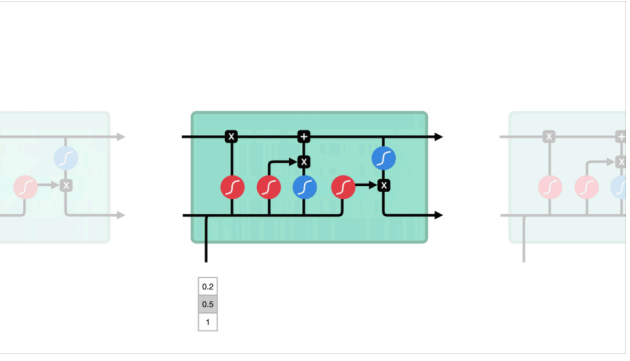


Illustrated Guide to LSTM's and GRU's: A step by step explanation

Michael Phi · Fullstack Engineer in Towards Data Science · 15 min read · Sep 24, 2018



Hi and welcome to an Illustrated Guide to Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU). I'm Michael, and I'm a Machine Learning Engineer in the AI voice assistant space.

In this post, we'll start with the intuition behind LSTM's and GRU's. Then I'll explain the internal mechanism that allow LSTM's and GRU's to perform so well. If you want to understand what's happening under the hood for these two networks, then this post is for you.

You can also watch the video version of this post on youtube if you prefer.

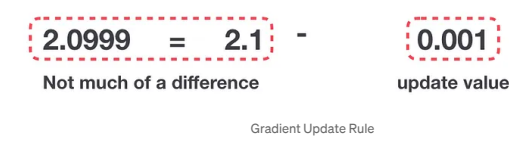


The Problem, Short-term Memory

Recurrent Neural Networks suffer from short-term memory. If a sequence is long enough, they'll have a hard time carrying information from earlier time steps to later ones. So if you are trying to process a paragraph of text to do predictions, RNNs may lose important information from the beginning.

During back propagation, recurrent neural networks suffer from the vanishing gradient problem. Gradients are values used to update a neural network's weights. The vanishing gradient problem is when the gradient shrinks as it back-propagates through time. If a gradient value becomes extremely small, it doesn't contribute too much learning.

new weight = weight - learning rate*gradient

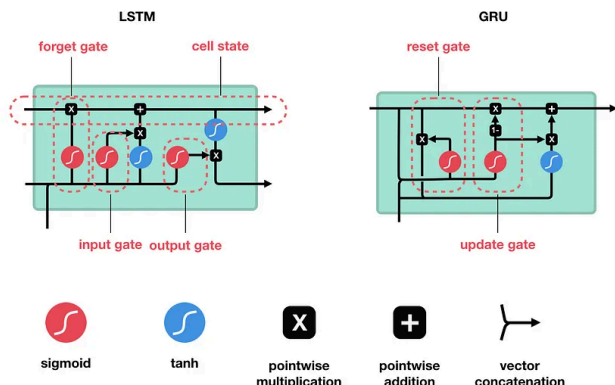


So in recurrent neural networks, layers that get a small gradient update stop learning. These are usually the earlier layers. So because these layers don't learn, RNNs can forget what it even in longer sequences, thus having a short-term memory. If you want to know more about the mechanics of recurrent neural networks in general, you can read my previous post here.

Illustrated Guide to Recurrent Neural Networks
Hi and welcome to an Illustrated Guide to recurrent neural networks. In this tutorial, we'll explore the internal mechanism that allow LSTM's and GRU's to perform so well. If you want to understand what's happening under the hood for these two networks, then this post is for you.

LSTM's and GRU's as a solution

LSTM's and GRU's were created as the solution to short-term memory. They have internal mechanisms called gates that can regulate the flow of information.

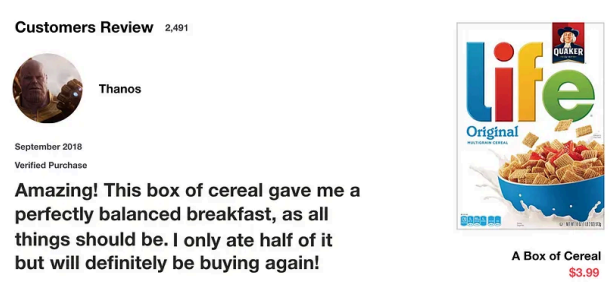


sequences to make predictions. Almost all state-of-the-art results based on recurrent neural networks are achieved with these two networks. LSTM's and GRU's can be found in speech recognition, speech synthesis, and text generation. You can even use them to generate captions for videos.

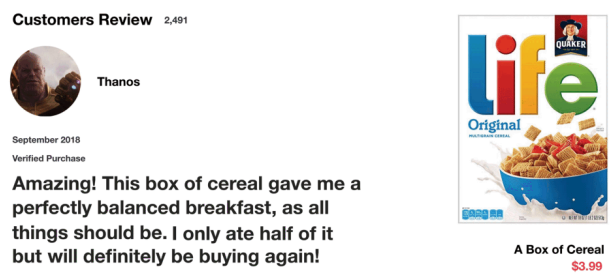
Ok, so by the end of this post you should have a solid understanding of why LSTM's and GRU's are great at processing long sequences. I am going to approach this with intuitive explanations and illustrations and avoid as much math as possible.

Intuition

Ok, let's start with a thought experiment. Let's say you're looking at reviews online to determine if you want to buy Life cereal. You don't know why you'll first read the review then determine if someone thought it was good or if it was bad.



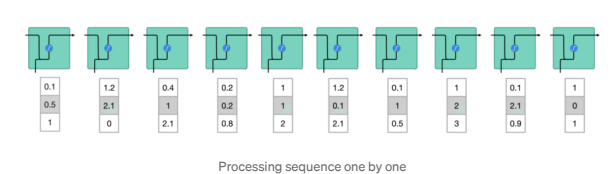
When you read the review, your brain subconsciously only remembers important keywords. You pick up words like "Amazing" and "perfectly balanced breakfast". You don't care much for words like "this", "gave", "at", "bought", etc. If a friend asks you the next day what the review said, you probably wouldn't remember it word for word. You might remember the main points though like "will definitely be buying again". If you're a lot like me, the other words will fade away from memory.



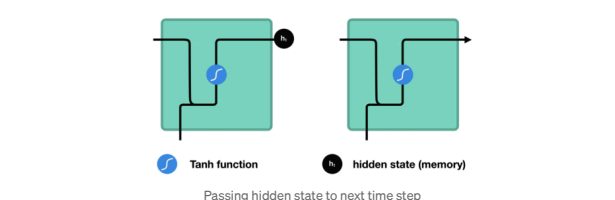
And that is essentially what an LSTM or GRU does. It can learn to keep only relevant information to make predictions, and forget irrelevant data. In this case, the words you remembered made you judge that it was good.

Review of Recurrent Neural Networks

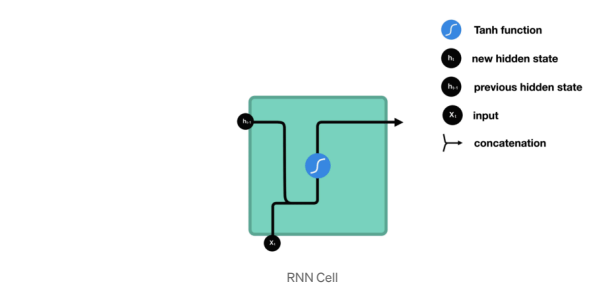
To understand how LSTM's or GRU's achieve this, let's review the recurrent neural network. An RNN works like this: First words get transformed into machine-readable vectors. Then the RNN processes the sequence of vectors one by one.



While processing, it passes the previous hidden state to the next step of the sequence. The hidden state acts as the neural network's memory. It holds information on previous data the network has seen before.

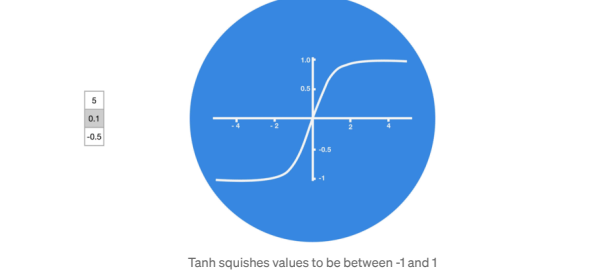


Let's look at a cell of the RNN to see how you would calculate the hidden state. First, the input and previous hidden state are combined to form a vector. That vector now has information on the current input and previous inputs. The vector goes through the tanh activation, and the output is the new hidden state, or the memory of the network.

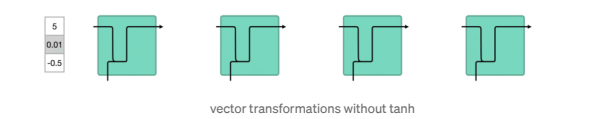


Tanh activation

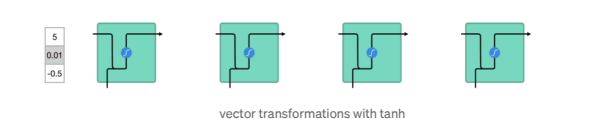
The tanh activation is used to help regulate the values flowing through the network. The tanh function squashes values to always be between -1 and 1.



When vectors are flowing through a neural network, it undergoes many transformations due to various math operations. So imagine a value that continues to be multiplied by let's say 3. You can see how some values can explode and become astronomical, causing other values to seem insignificant.



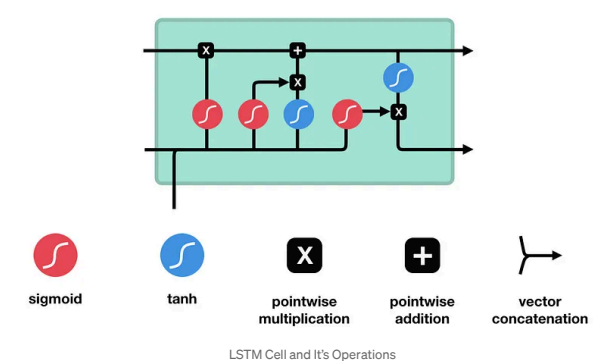
A tanh function ensures that the values stay between -1 and 1, thus regulating the output of the neural network. You can see how the same values from above remain between the boundaries allowed by the tanh function.



So that's an RNN. It has very few operations internally but works pretty well given the right circumstances (like short sequences). RNNs have a low computational resource than its related vectors, LSTM's and GRU's.

LSTM

An LSTM has a similar control flow as a recurrent neural network. It processes data passing on information as it propagates forward. The differences are the operations within the LSTM's cells.



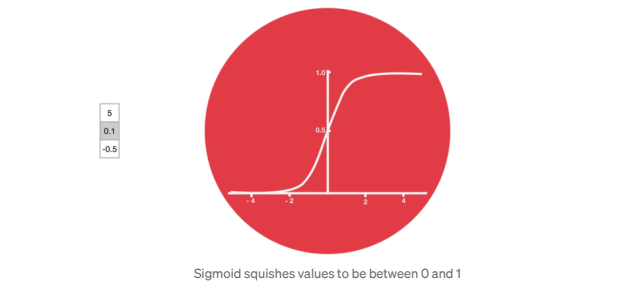
These operations are used to allow the LSTM to keep or forget information. Now looking at these operations can get a little overwhelming so we'll go over this step by step.

Core Concept

The core concept of LSTM's are the cell state, and it's various gates. The cell state act as a transport highway that transfers relative information all the way down the sequence chain. You can think of it as the "memory" of the network. The cell state, in theory, can carry relevant information throughout the processing of the sequence. In even information from the earlier time steps can make it's way to later time steps, reducing the effects of short-term memory. As the cell state goes on its journey, information gets added or removed to the cell state as gates. The gates are different neural networks that decide which information is allowed on the cell state. The gates can learn when information is relevant to keep or forget during training.

Sigmoid

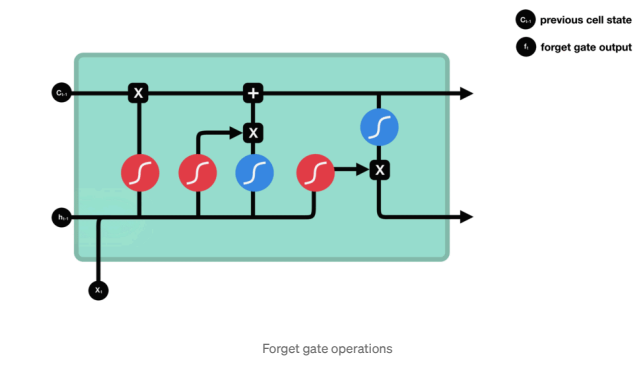
Sigmoid contains sigmoidal activations. A sigmoid activation is similar to the tanh activation. Instead of squishing values between -1 and 1, it squashes values between 0 and 1. That is helpful to update or forget data because any number getting multiplied by 0 is 0, causing values to disappear or be "forgotten." Any number multiplied by 1 is the same value therefore that value stays the same or is "kept." The network can learn which data is not important therefore can be forgotten or which data is important to keep.



Let's dig a little deeper into what the various gates are doing, shall we? So we have three different gates that regulate information flow in an LSTM cell. A forget gate, input gate, and output gate.

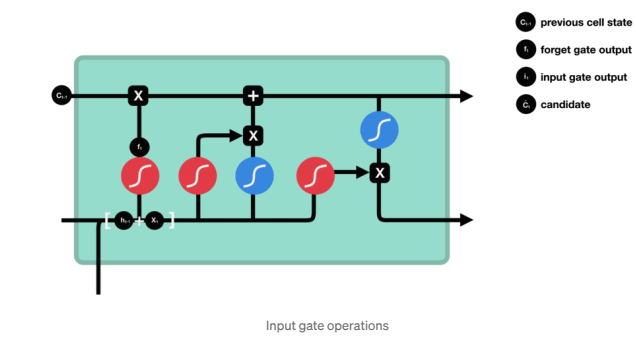
Forget gate

First, we have the forget gate. This gate decides what information should be thrown away or kept. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.



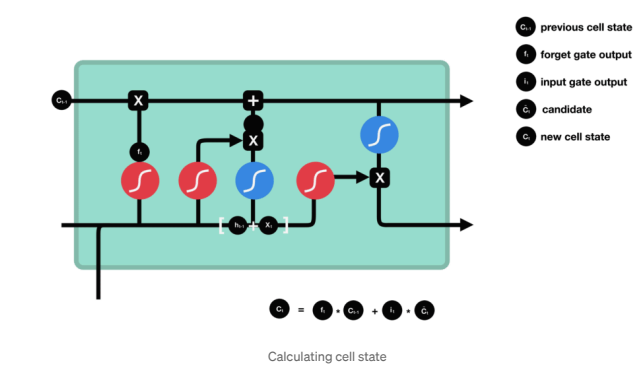
Input Gate

To update the cell state, we have the input gate. First, we pass the previous hidden state and current input into a sigmoid function. That decides which values will be updated by transferring the values to be between 0 and 1. 0 means not important, and 1 means important. You also pass the hidden state and current input into the tanh function to squish values between -1 and 1 to help regulate the network. Then you multiply the tanh output with the sigmoid output. The sigmoid output will decide which information is important to keep from the tanh output.



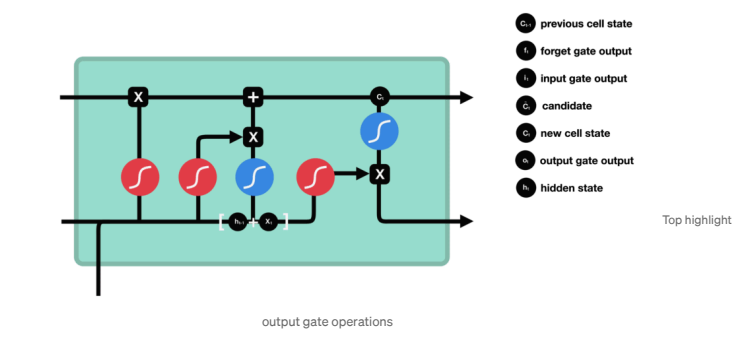
Cell State

Now we should have enough information to calculate the cell state. First, the cell state gets pointwise multiplied by the forget vector. This has a possibility of dropping values in the cell state if it gets multiplied by values near 0. Then we take the output from the input gate and do a pointwise addition which updates the cell state to new values that the neural network finds relevant. That gives us our new cell state.



Output Gate

Last we have the output gate. The output gate decides what the next hidden state should be. Remember that the hidden state contains information on previous inputs. The hidden state is also used for predictions. First, we pass the previous hidden state and the current input into a sigmoid function. Then we pass the newly modified cell state to the tanh function. We multiply the tanh output with the sigmoid output to decide what information the hidden state should carry. The output is the hidden state. The new cell state and the new hidden is then carried over to the next time step.



To review, the Forget gate decides what is relevant to keep from prior steps. The input gate decides what information is relevant to add from the current step. The output gate determines what the next hidden state should be.

Code Demo

For those of you who understood better through seeing the code, here is an example using python pseudo code.

```
def LSTMCell(prev_ct, prev_ht, input):
    combine = prev_ht + input
    ft = forget_layer(combine)
    candidate = candidate_layer(combine)
    it = forget_layer(combine)
    ct = prev_ct + ft * candidate
    ht = output_layer(combine)
    ht = ht * tanh(ct)
    return ht, ct

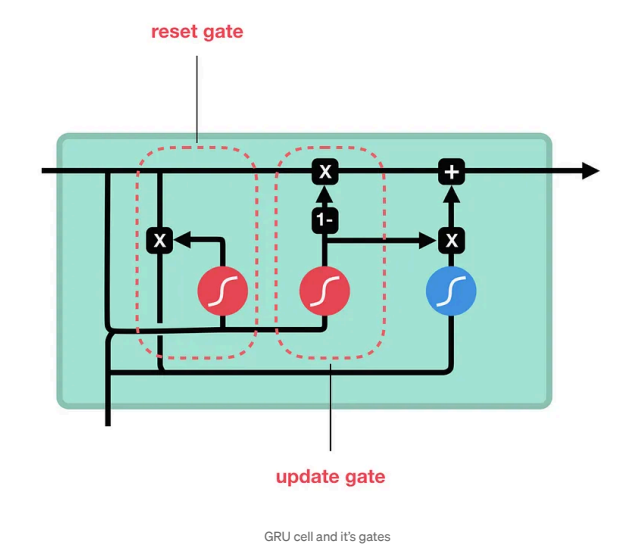
ct = [0, 0, 0]
ht = [0, 0, 0]
for input in inputs:
    ct, ht = LSTMCell(ct, ht, input)
```

- python pseudo code
1. First, the previous hidden state and the current input get concatenated. We'll call it combine.
 2. Combine gets fed into the forget layer. This layer removes non-relevant data.
 3. A candidate layer is created using combine. The candidate holds possible values to add to the cell state.
 4. A candidate layer is created using combine. The candidate holds possible values to add to the cell state.
 5. Combine also gets fed into the input layer. This layer decides what data from the candidate should be added to the new cell state.
 6. After computing the forget layer, candidate layer, and the input layer, the cell state is calculated using those vectors and the previous cell state.
 7. Pointwise multiplying the output and the new cell state gives us the new hidden state.

That's let the control flow of an LSTM network are a few tensor operations and a for loop. You can use the hidden states for predictions. Combining all those mechanisms, an LSTM can choose which information is relevant to remember or forget during sequence processing.

GRU

So now we know how an LSTM work, let's briefly look at the GRU. The GRU is the newer generation of Recurrent Neural networks and is pretty similar to an LSTM. GRUs get rid of the cell state and used the hidden state to transfer information. It also only has two gates, a reset gate and update gate.



Update Gate

The update gate acts similar to the forget and input gate of an LSTM. It decides what information to throw away and what new information to add.

Reset Gate

The reset gate is another gate is used to decide how much past information to forget.

And that's a GRU. GRUs has fewer tensor operations, therefore, they are a little speedier to train than LSTMs. There isn't a clear winner which one is better. Researchers and engineers usually try both to determine which one works better for their use case.

So That's It

To sum this up, RNNs are good for processing sequence data for predictions but suffer from short-term memory. LSTMs and GRUs were created as a method to mitigate short-term memory using mechanisms called gates. Gates are just neural networks that regulate the flow of information flowing through the sequence chain. LSTMs and GRUs are used in state of the art deep learning applications like speech recognition, speech synthesis, natural language understanding, etc.

If you're interested in going deeper, here are links of some fantastic resources that can give you a different perspective in understanding LSTMs and GRUs. This post was heavily inspired by them.

<https://www.worldatlas.com/2015/04/recurrent-neural-network-intro-part-1.html#introduction-a-recurrent-run-with-gates-and-their>

<http://yoshiki.vishnu.kr/2015-08-Understanding-LSTM/>

<https://www.worlde.com/2016-07/WCNDP-REV>

I had a lot of fun making this post so let me know in the comments if this was helpful or what you would like to see in the next one. And as always, thanks for reading!

Check out michaelphi.com for more content like this.

Want more Content? Check out my blog or <https://www.michaelphi.com>

Like to watch project-based videos? Check out my [Youtube!](#)

Stay up to date on articles and videos by signing up for my [email newsletter!](#)

Machine Learning Deep Learning Artificial Intelligence LSTM Recurrent Networks

Published in Towards Data Science
179 Followers Last published 1 year ago
Your favorite for data science. A publication sharing insights, stories and codes.

Written by Michael Phi
Full Stack Engineer Data Scientist
Software and Machine Learning Research Engineer. He is a machine learning engineer and data scientist who is currently working on AI, deep learning, and data science.

Responses (116)

What are your thoughts?

Thapa A
2024-11-23 1:00
Amazing! You explained not only the concepts, intuition, and how the ability to explain things helps deeply. Still for waiting for your next post!

Like 0 1 reply Reply

Ujjal Bhatia
2024-11-23 1:00
Hi, the examples are impressive and very easy to understand. Thank you very much. BTW, the visualization is amazing and can you show how you do the work with visualization of what you are doing?

Like 0 1 reply Reply

Deepak Datta
2024-11-23 1:00
Amazing explanation. Thank you!

Like 0 1 reply Reply

See all responses

More from Michael Phi and Towards Data Science

Illustrated Guide to Transformers
The ultimate guide to understanding the Transformer architecture. This book is a comprehensive guide to the Transformer architecture, covering everything from the basics to the latest research. It is a must-read for anyone interested in deep learning and natural language processing.

May 1, 2020 12 min 10 10 10

LLM Building - Intuitively and Exhaustively Explained
The ultimate guide to understanding the LLM architecture. This book is a comprehensive guide to the LLM architecture, covering everything from the basics to the latest research. It is a must-read for anyone interested in deep learning and natural language processing.

May 1, 2020 12 min 10 10 10

Illustrated Guide to LSTM's and GRU's: A step by step explanation | by Michael Phi | Towards Data Science

in Towards Data Science by James Hines

Another Hike Up Everest
How to make progress on hard problems in AI

10/24
17k
14
5

12

Illustrated Guide to Recurrent Neural Networks

Understanding the intuition

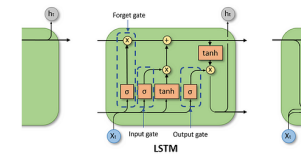
10/23/2018
17.1k
14
28

12

See all from Michael Phi

See all from Towards Data Science

Recommended from Medium



in Towards Data Science by James Hines

A Brief Introduction to Recurrent Neural Networks
An introduction to RNN, LSTM, and GRU and their implementations

Nov 15, 2022
10.8k
14
5

12

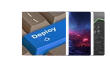


in StackShare by Mike Mermel

Python is No More The King of Data Science
8 Reasons Why Python is Losing its Crown

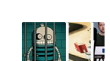
Oct 15
10.4k
14
30

12



Predictive Modeling w/ Python
100 lessons - 400 pages

10/20/2020



AI Population
100 lessons - 400 pages

10/20/2020



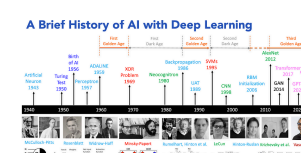
Natural Language Processing
100 lessons - 400 pages

10/20/2020



Practical Guides to Machine Learning
100 lessons - 400 pages

10/20/2020

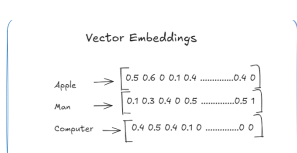


A Brief History of AI with Deep Learning

Artificial intelligence (AI) and deep learning have seen remarkable progress over the years.

Deep 1
10.8k
14
5

12



Vector Embedding

Math:
$$\text{word} \rightarrow \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{bmatrix}$$

vec:
$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{bmatrix}$$

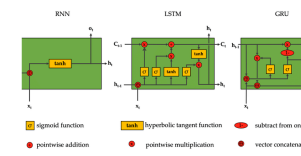
matrix:
$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \\ 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \\ 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \end{bmatrix}$$

in Towards AI by MadelonMabey

A Complete Guide to Embedding For RLP & Generative AIGLM
Understand the concept of vector embedding, why it's needed, and...

Oct 19
10.4k
14
10

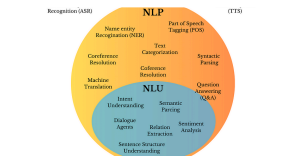
12



RNN vs. LSTM vs. GRU: A Comprehensive Guide to...
Introduction

Jul 5
10.8k
14
5

12



LLM Architectures Explained: MLP Fundamentals Part 1
Deep Dive into the architecture & building of neural networks using MLP.

Aug 15
10.8k
14
10

12

See more recommendations

Help Status About Contact Privacy Policy Privacy Terms Text to speech Terms

https://towardsdatascience.com/illustrated-guide-to-lstm-and-gru-s-a-step-by-step-explanation-44e9eb85d21

3/3